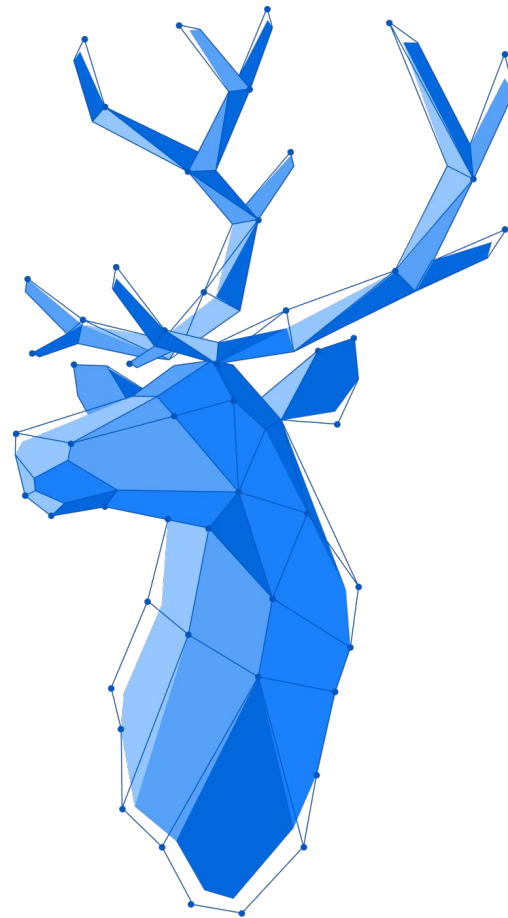




Dependency Injection

Von Gerald Koch



Spring Bean Lebenszyklus



Instantiate



(Re)Use



Collect

- Verwaltet und Instantiiert `@Component/@Bean` (und ableitend) annotierte Klassen und stellt sie bereit
- Abhängigkeiten werden über einen Abhängigkeitsbaum aufgelöst
- Konstruktoraufrufe sollten vermieden werden (von Spring verwaltet). Das verhindert Memory Leaks.
- Statische Konstruktorparameter müssen über Setter eingegeben werden
- Beans können verschiedene Scopes haben

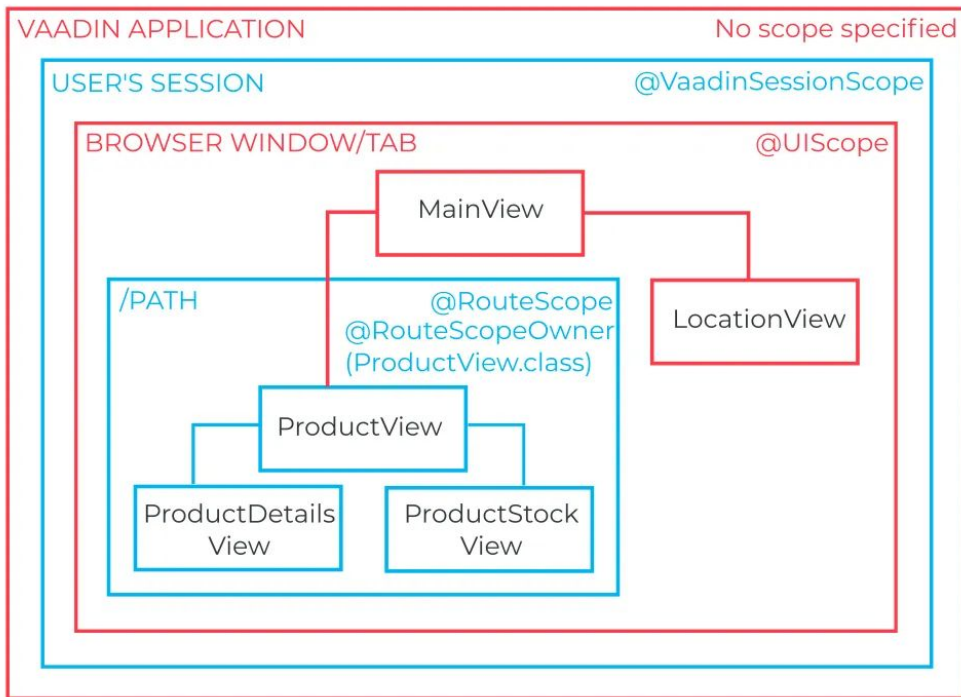
Dependency Injection

Spring – Scopes

- Singleton Scope (Nur eine Instanz)
 - Keine zusätzliche Scope Annotation
 - `@Scope("singleton")`
 - `@Scope(value = ConfigurableBeanFactory.SCOPE_SINGLETON)`
- Prototype Scope (Neue Instanz bei jeder Injection)
 - `@Scope("prototype")`
 - `@Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE)`
- Session Scope (Eine Instanz pro Session)
 - `@SessionScope`
 - `@Scope(value = WebApplicationContext.SCOPE_SESSION, proxyMode = ScopedProxyMode.TARGET_CLASS)`
- Mehr Scopes: <https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html>

Dependency Injection

Vaadin – Scopes



- Vaadin Application (kein Scope) entspricht dem Singleton Scope
- `@VaadinSessionScope` liegt innerhalb von Spring `@SessionScope`

Mehr Information:

<https://vaadin.com/docs/latest/flow/integrations/spring/scopes>

Hierarchie von Scopes

```
@UIScope //one per Vaadin UI
public class User {

    private String name;
    private String email;
    private String address;
}
```

```
@Service //singleton scope
public class UserService {

    private User user;

    public Presenter (User user) { // which one?
        this.user = user;
    }
}
```

- Kleinere Scopes können problemlos auf größere Scopes über Injections zugreifen
- Größere Scopes benötigen Auswahlhilfe um die richtige Instanz kleinerer Scopes zu bekommen (Außer Instanzen mit Prototype Scope)
- Proxy nötig für Injections von Klassen kleinerer Scopes in Klassen größerer Scopes

Scopes Proxies

- Proxies ermöglichen es den größeren Scopes den Zugriff auf Instanzen kleinerer Scopes
- Proxies werden automatisch vom Spring Framework erstellt, falls konfiguriert
- Welcher Typ von Proxy verwendet werden soll, wird über die Eigenschaft `proxyMode` eingestellt
 - `@Scope(value = XYZ, proxyMode = ScopedProxyMode.TARGET_CLASS)`
- Es gibt verschiedene Proxy Modi:
 - **DEFAULT:** Verwendet den global eingestellten Proxy Modus
 - **NO:** Verwendet keinen Proxy
 - **TARGET_CLASS:** Verwendet die CGLIB Bibliothek um eine Proxyklasse aus der eigentlichen Klasse abzuleiten
 - **INTERFACES:** Verwendet das injizierte Interface, um eine Proxy Klasse zu erstellen

Achtung: Es dürfen nur Interfaces referenziert werden
- Vaadin Scopes haben keine Proxies konfiguriert
- Spring Scopes, die nicht Singleton bedeuten, haben `TARGET_CLASS` konfiguriert

Spring – Value Injection

- `@Value(<value id>)`
 - Kann mit der Property Placeholder Syntax verwendet werden `"${...}"`
 - Kann mit Spring Expression Language (SpEL) verwendet werden `"#{${...}}"`
- Property Placeholder Syntax:
 - Ohne Default-Wert: `${property}`
 - Mit Default-Wert: `${property:default}`
 - Funktioniert mit String und primitiven Datentypen und deren Object Wrappers
 - Funktioniert mit Listen oben genannter Typen*: `(prop=value1,value2,value3 )`
- Spring Expression Language
 - Ohne Default Wert (Wert nötig): `#{${property}}`
 - Mit default (Wert optional): `#{${property:default}}`
 - Unterstützt Maps `(prop={key1:'value1', key2:'value2', ...} )`

Mit Default-Wert ist es immer besser

Spring – Best Practices

- So stateless wie möglich (Klassenvariablen auf ein Minimum reduzieren) ganz besonders in der Repository- und Service-Schicht
- Möglichst immer im Konstruktor injizieren und `@Autowired` an Klassenvariablen vermeiden (verhindert zyklische Referenzen und Memory Leaks)
- UI Komponenten: Layout und Komponenten im Konstruktor erzeugen und Methoden wie `setItems(...)` oder `setDataProvider(...)` benutzen, um Daten einzugeben (Siehe Vaadin Klasse Grid)
- Spring nicht in statischem Kontext benutzen
- Verwendung von Konstruktoren vermeiden! Spring erzeugt die Instanzen. Selbstkonstruierte Instanzen verlieren den Spring Kontext
- Möglichst immer Defaults bei `@Value` Annotationen verwenden